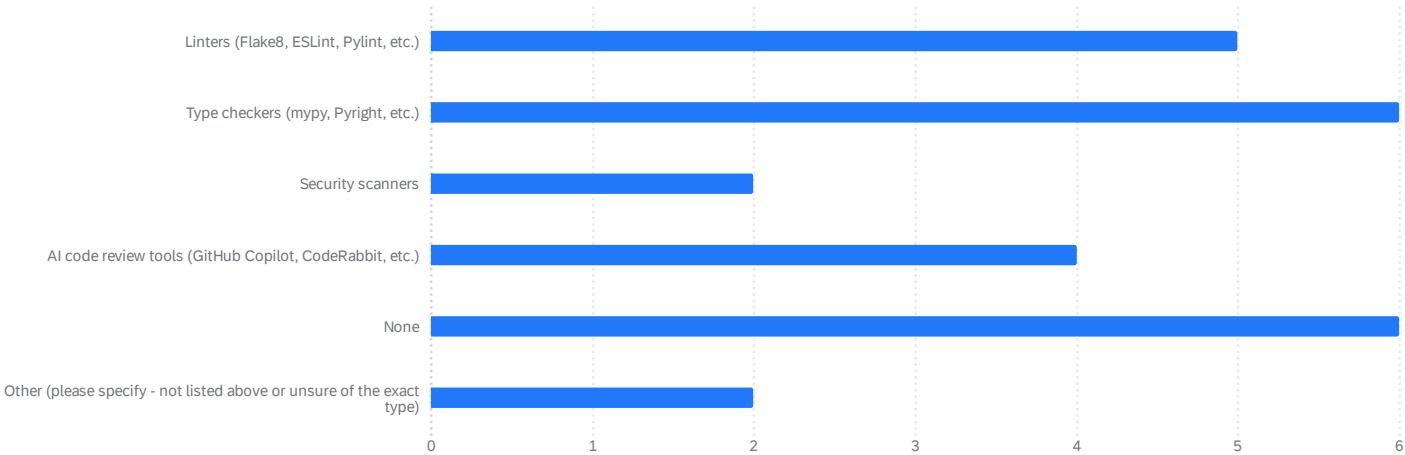


Project Tooling: Do you currently use any static analysis tools in the repository [Field-rep... 14 ⓘ



Project Tooling: Do you currently use any static analysis tools in the repository [Field-rep... 14 ⓘ

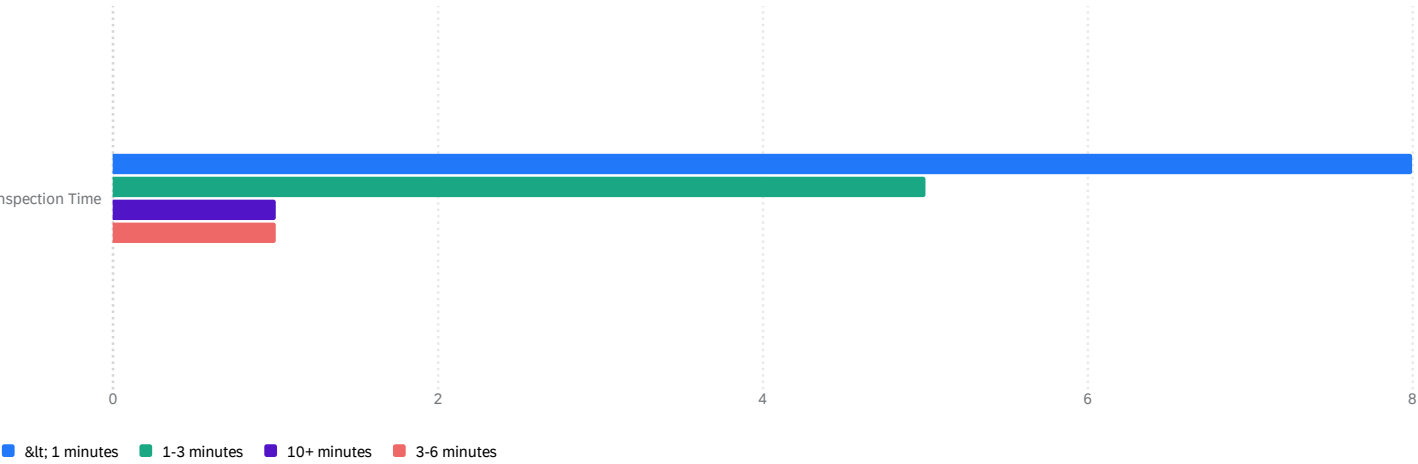
Project Tooling - Do you currently use any static analysis tools in the repository [Field-repo_name]? Please select all that apply. - Selected Choice	Count	Count
Linters (Flake8, ESLint, Pylint, etc.)	36%	5
Type checkers (mypy, Pyright, etc.)	43%	6
Security scanners	14%	2
AI code review tools (GitHub Copilot, CodeRabbit, etc.)	29%	4
None	43%	6
Other (please specify - not listed above or unsure of the exact type)	14%	2

Project Tooling: Do you currently use any static analysis tools in the repository [Field-rep...: Other (please specify - not listed above or unsure of the exact type) - ⓘ
Text 15

Other (please specify)

Mend Checks

Invest Time: How long did you spend inspecting this warning? 15



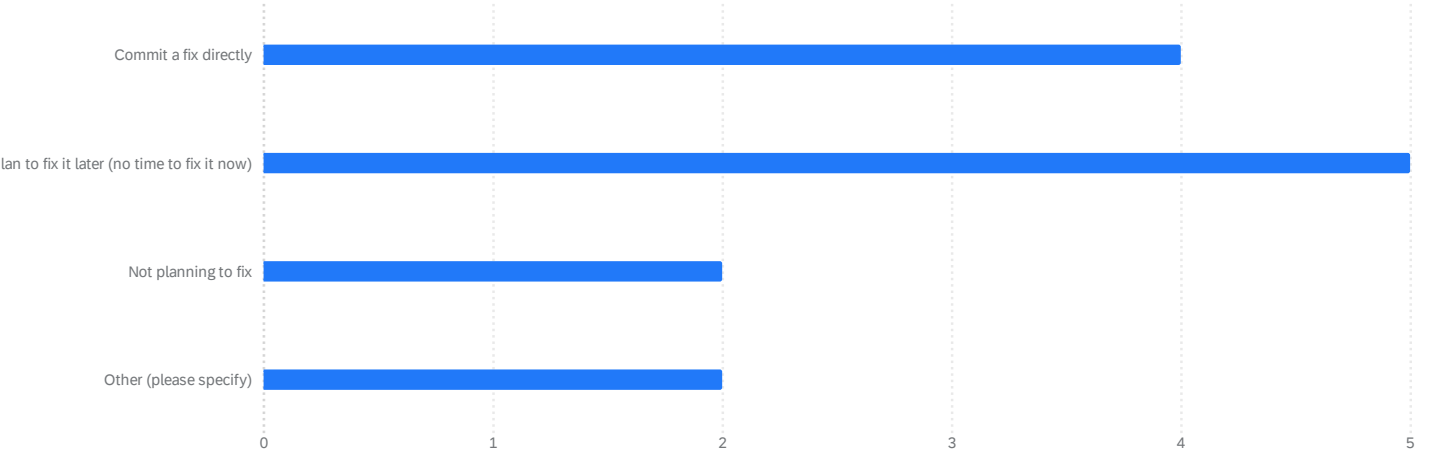
Invest Time: How long did you spend inspecting this warning? 15

How long did you spend inspecting this warning?	Average (How long did you spend inspecting this warning?)	Minimum (How long did you spend inspecting this warning?)	Maximum (How long did you spend inspecting this warning?)	Standard Deviation (How long did you spend inspecting this warning?)	Variance (How long did you spend inspecting this warning?)	Count
Inspection Time	1.73	1.00	5.00	1.06	1.13	15

To Address?: Do you think this warning represents an issue in the codebase that should be fixed? 15 ⓘ

To Address? - Do you think this warning represents an issue in the codebase that should be fixed?	Count	Count
Yes	7%	1
Not necessary, but nice to fix	80%	12
No	13%	2

Fixing: What action would you take (or plan to take) to address this warning? 13 ⓘ



Fixing: What action would you take (or plan to take) to address this warning? 13 ⓘ

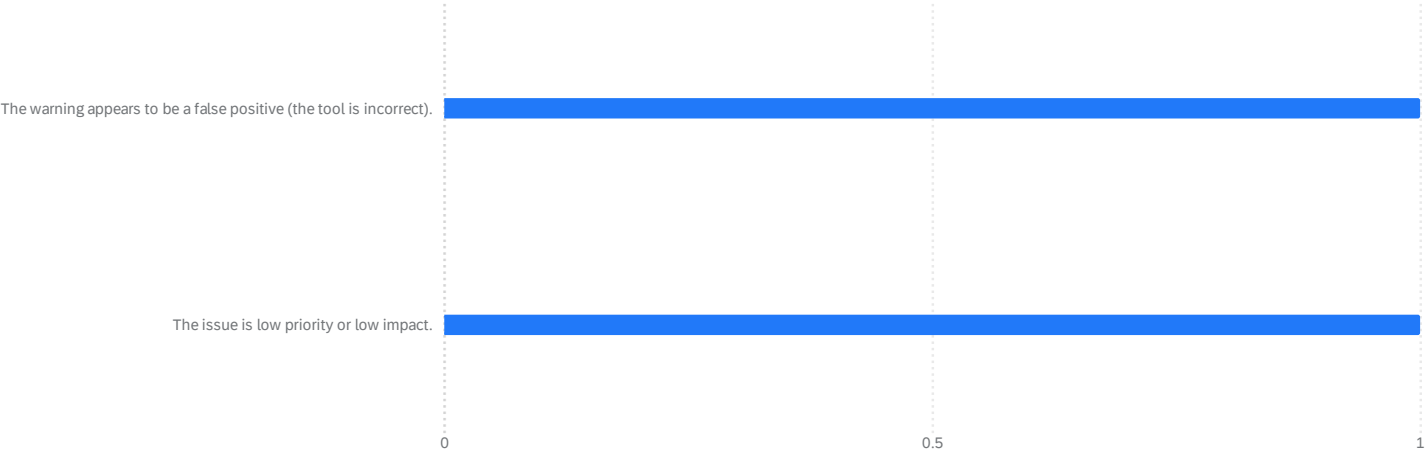
Fixing - What action would you take (or plan to take) to address this warning? - Selected Choice	Count	Count
Commit a fix directly	31%	4
Plan to fix it later (no time to fix it now)	38%	5
Not planning to fix	15%	2
Other (please specify)	15%	2

Fixing: What action would you take (or plan to take) to address this warning?: Other (please specify) - Text 15 ⓘ

Other (please specify)

Hey, So this will be quite a big answer and i do not understand how formatting works here so apologies if it comes a bit messy, To address the warning regarding test reproducibility, I would introduce deterministic random seed initialization within the test suite. Specifically, I would add `random.seed(42)` to the `setUp()` method of each test class in `test_genetic_algorithm.py`, ensuring that every test begins with a consistent random state. Since genetic algorithms inherently rely on random operations such as population initialization, crossover, mutation, and selection, test outcomes can vary between executions if the random number generator is not controlled. By seeding the random module at the start of each test, the same sequence of random values is generated every time, resulting in predictable and repeatable test behavior. This change would be applied across all relevant test classes, including `TestIndividual`, `TestFitnessFunctions`, `TestGAConfig`, `TestGeneticAlgorithm`, `TestCrossoverOperations`, `TestMutationOperations`, `TestUtilityFunctions`, and `TestSelectionMethods`. In addition to implementing the seed initialization, I would document the purpose of the seed through clear comments within the test code, explaining that it exists to ensure reproducible and deterministic test results across multiple runs and environments. If the project later transitions to or adopts `pytest`, an alternative approach would be to use the `pytest-randomly` plugin, which automatically manages random seeds and provides additional control over test reproducibility. However, for the current `unittest`-based setup, adding explicit seed initialization is the simplest and most maintainable solution. The effort required for this improvement is minimal, likely taking no more than 10–15 minutes, and it can be completed during the next test maintenance cycle. Although the issue is not critical, implementing this change follows established testing best practices and improves the reliability of the test suite. After the modification, I would verify the solution by executing the test suite multiple times to confirm that results remain identical across runs and by ensuring that test execution order does not introduce any unintended state leakage. The change could then be documented with a commit message such as “Add random seed initialization to tests for reproducibility.” This approach is intentionally pragmatic and non-invasive. It affects only the test code and has no impact on production functionality, making it a low-risk enhancement. At the same time, it directly addresses the root cause of non-deterministic test behavior without removing randomness from the genetic algorithm itself, which would

Not Fixing: Why do you think this warning should not be addressed (or why are you unsur... 2 ⓘ



Not Fixing: Why do you think this warning should not be addressed (or why are you unsur... 2 ⓘ

Not Fixing - Why do you think this warning should not be addressed (or why are you unsure)? Please select all that apply. - Selected Choice	Count	Count
The warning appears to be a false positive (the tool is incorrect).	50%	1
The issue is low priority or low impact.	50%	1

Allow Invest Time: How long would you typically be willing to spend investigating a warning with the same severity as the warning shown previously? 15 ⓘ

Allow Invest Time - How long would you typically be willing to spend investigating a warning with the same severity as the warning shown previously?	Count	Count
< 1 minutes	20%	3
1-3 minutes	27%	4
3-6 minutes	20%	3
6-10 minutes	13%	2
10+ minutes	7%	1
Depends on severity	13%	2

Allow Invest Time: How long would you typically be willing to spend investigating a warning with the same severity as the warning shown previously? 15 ⓘ

Average (Allow Invest Time - How long would you typically be willing to spend investigating a warning with the same severity as the warning shown previously?)	2.93
Minimum (Allow Invest Time - How long would you typically be willing to spend investigating a warning with the same severity as the warning shown previously?)	1.00
Maximum (Allow Invest Time - How long would you typically be willing to spend investigating a warning with the same severity as the warning shown previously?)	6.00
Standard Deviation (Allow Invest Time - How long would you typically be willing to spend investigating a warning with the same severity as the warning shown previously?)	1.53
Variance (Allow Invest Time - How long would you typically be willing to spend investigating a warning with the same severity as the warning shown previously?)	2.33
Count	15

More Information: In addition to the information we provided previously, what additional information would help you better understand, investigate, or fix such a warning reported by our dynamic analysis tools? 15 ⓘ

In addition to the information we provided previously, what additional info...

The single most useful addition would be data-flow / call-stack context: the warning points at line 63 (random.randint inside the generate_test_data helper), but every caller already sets a fixed seed beforehand (random.seed(42) at line 91, etc.). If the report showed the execution path leading to the flagged call — and whether a seed was set earlier on that path — I could have dismissed this as a false positive in seconds instead of opening the file and tracing callers manually.

Other things that would help:

- An inline code snippet around the flagged line (and the relevant caller), not just a link.
- Why the tool concluded no seed was active — what state it observed at runtime.
- Whether the random values actually flow into an assertion (i.e., does non-determinism here actually affect pass/fail?), since that determines real impact.
- A confidence/severity rationale and a concrete suggested fix or diff.
- For reproducibility warnings specifically: recognition that per-test seeding (already used here) is a valid, often preferable alternative to a single global session seed.

To make the warning significantly more actionable and valuable for developers, additional supporting information should accompany the finding. The most important enhancement would be the inclusion of concrete evidence demonstrating the problem in practice. For example, providing before-and-after test outputs, examples of flaky test failures, or statistics showing inconsistent outcomes across multiple test executions would transform the warning from a theoretical concern into a measurable issue. Information about how the warning was detected would also be beneficial, including whether the analysis tool observed non-deterministic behavior directly, the confidence level of the finding, and the specific tests impacted.

History Violations: The dashboard above shows that some warnings detected by our dynamic analysis tools remain unfixed for extended periods of time. What do you think are the main reasons for this? 15 ⓘ

According to our dashboard (view dashboard), many violations detected by ou...

- Not ignored — never seen, and on a repo I'm no longer actively developing.
- Honestly, Havent had time
- warnings are modeled wrong: can't separate signal from noise because they're asking the wrong question. The question isn't whether to ignore the warning; it's what the user intends about the correct shape of the code. Knowing that will tell whether a warning needs to be resolved by fixing or by suppressing. THAT is actionable. Long lists of warnings are not.
- They are not critical in my perspective.
- Lack of resources

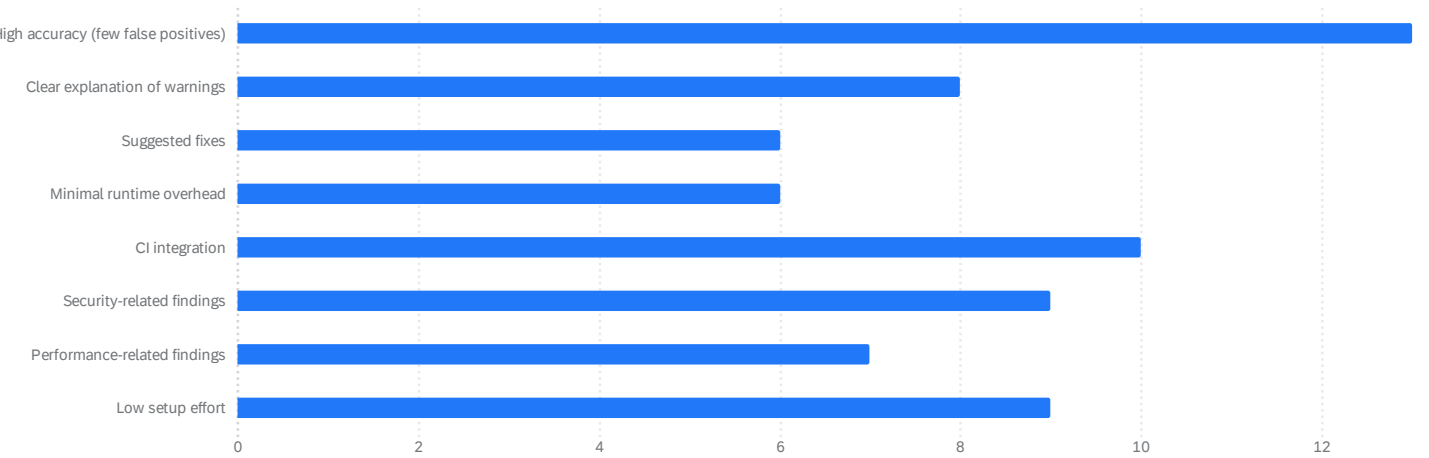
Acceptable Overhead: Based on the results above for monitoring project code only, would this runtime overhead be acceptable in your CI workflow? 13 ⓘ

Acceptable Overhead - Based on the results above for monitoring project code only, would this runtime overhead be acceptable in your CI workflow?	Count	Count
Up to 2× runtime is acceptable	54%	7
Up to 5× runtime is acceptable	23%	3
Up to 10× runtime is acceptable	8%	1
Up to 25× runtime is acceptable	8%	1
Any slowdown is acceptable	8%	1

Acceptable Overhead: Based on the results above for monitoring project code only, would this runtime overhead be acceptable in your CI workflow? 15 ⓘ

Average (Acceptable Overhead - Based on the results above for monitoring project code only, would this runtime overhead be acceptable in your CI workflow?)	9.77
Minimum (Acceptable Overhead - Based on the results above for monitoring project code only, would this runtime overhead be acceptable in your CI workflow?)	6.00
Maximum (Acceptable Overhead - Based on the results above for monitoring project code only, would this runtime overhead be acceptable in your CI workflow?)	17.00
Standard Deviation (Acceptable Overhead - Based on the results above for monitoring project code only, would this runtime overhead be acceptable in your CI workflow?)	4.19
Variance (Acceptable Overhead - Based on the results above for monitoring project code only, would this runtime overhead be acceptable in your CI workflow?)	17.56
Count	15

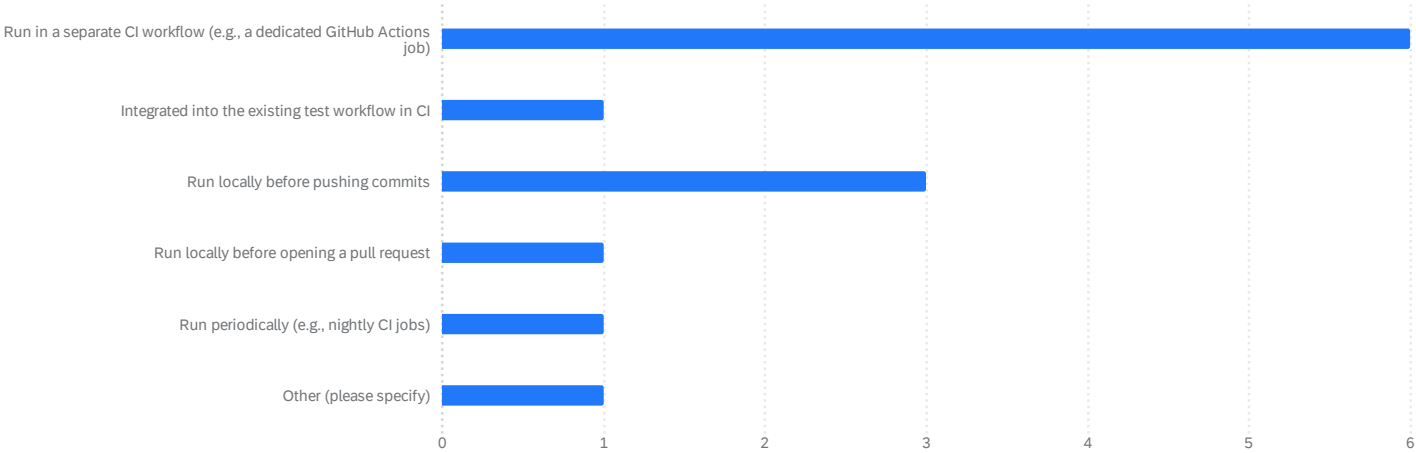
New Features: Which factors would matter to you if you were to adopt our dynamic analysis... 13 ⓘ



New Features: Which factors would matter to you if you were to adopt our dynamic analysis...

New Features - Which factors would matter to you if you were to adopt our dynamic analysis tools? Please select all that apply. - Selected Choice	Count	Count
High accuracy (few false positives)	100%	13
Clear explanation of warnings	62%	8
Suggested fixes	46%	6
Minimal runtime overhead	46%	6
CI integration	77%	10
Security-related findings	69%	9
Performance-related findings	54%	7

Integrations: If you were to adopt our dynamic analysis tools, how would you prefer to in...



Integrations: If you were to adopt our dynamic analysis tools, how would you prefer to in...

Integrations - If you were to adopt our dynamic analysis tools, how would you prefer to integrate them into your development workflow? - Selected Choice	Count	Count
Run in a separate CI workflow (e.g., a dedicated GitHub Actions job)	46%	6
Integrated into the existing test workflow in CI	8%	1
Run locally before pushing commits	23%	3
Run locally before opening a pull request	8%	1
Run periodically (e.g., nightly CI jobs)	8%	1
Other (please specify)	8%	1

Other (please specify)

Has to be local SPS pipeline

Any additional thoughts or feedback about these warnings or dynamic analysi...

Overall, the warning is useful because it highlights a class of issues that static analysis and code review often miss: problems that only become visible when code is executed under varying conditions. Dynamic analysis tools are particularly valuable for identifying flaky tests, non-deterministic behavior, resource leaks, performance regressions, race conditions, and environment-dependent failures. These issues can have a disproportionate impact on developer productivity because they erode trust in automated testing and make failures difficult to reproduce and debug.

That said, the effectiveness of these tools depends heavily on the quality and actionability of their findings. The most useful warnings are those that provide evidence, quantify impact, and clearly explain why the issue matters. Developers are far more likely to address a warning when they can see a failing example, understand the potential consequences, and have a clear remediation path. Conversely, warnings that only describe a theoretical risk without supporting data can contribute to alert fatigue, causing developers to ignore both low-value and high-value findings over time.

One area where these tools can provide additional value is prioritization. Not all warnings deserve equal attention, and developers benefit from understanding whether a finding is likely to cause production issues, CI instability, maintainability concerns, or simply represents a best-practice recommendation. Severity rankings should therefore consider factors such as reproducibility, observed failures, execution frequency, code coverage, and historical impact. A warning backed by observed flaky test behavior should generally receive a higher priority than one based solely on a static pattern match.

